



Лекція 17

ElasticSearch та Kibana

Elastic Stack (aka ELK)



Що таке Elasticsearch

Elasticsearch - розподілений рушій для пошуку даних і аналітики

- Можливість швидкого пошуку і побудова аналітичних запитів за довільними критеріями
- Повнотекстовий пошук
- Може служити в якості сховища даних (БД)
- Збереження і аналіз логів, подій
- Збереження і аналіз просторових (географічних) даних

Індекси і Документи

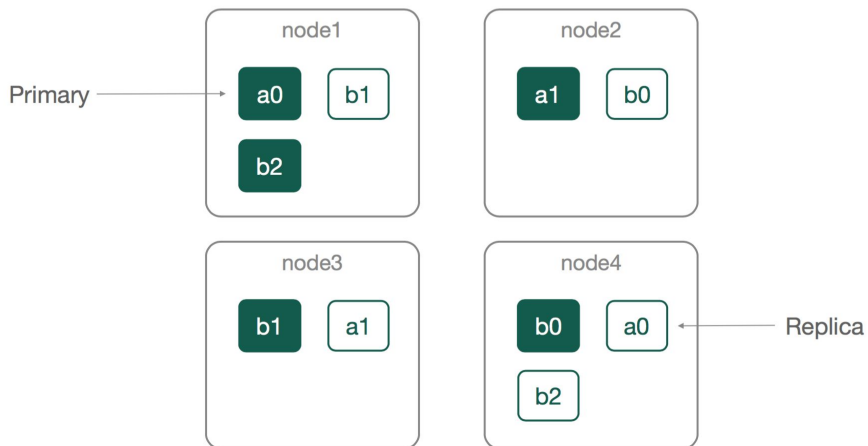
Index - зберігає документи в форматі JSON

Document - запис всередині індекса, в форматі JSON. Містить ідентифікатор (`_id`) та інформаційні поля.

В одному індексі зберігаються документи **одного типу** (з спільною структурою даних, яку визначає **Mapping**).

Shards and Replicas

A Cluster



Шарди та репліки: a0, a1; b0, b1, b2

Поля і типи даних

Field - поле документу, може бути простого типу, масив або вкладений документ

- String
 - Text
 - Keyword
- Numeric
 - long, integer, short, byte, double, float
- Date
- Boolean
- Nested

Mapping

Задає **схему** для індексу

Визначає назву полів, їх типи, а ще додаткові атрибути

- наприклад, формат для поля Date

Створюється автоматично при додаванні нових полів, або **явно** перед першим додаванням поля (наприклад, при створенні індексу)

Запускаємо Elasticsearch локально

1. Ставимо [Docker Desktop](#)

2. Створюємо текстовий файл `docker-compose.yml`

```
version: "3.9"
```

```
services:
```

```
  elasticsearch:
```

```
    image: elasticsearch:8.6.1
```

```
    container_name: elasticsearch
```

```
    environment:
```

- discovery.type=single-node
- ES_JAVA_OPTS=-Xms1g -Xmx1g
- xpack.security.enabled=false

```
    volumes:
```

- ./data:/usr/share/elasticsearch/data

```
    ports:
```

- 9200:9200

```
  kibana:
```

```
    image: kibana:8.6.1
```

```
    container_name: kibana
```

```
    ports:
```

- 5601:5601

```
    depends_on:
```

- elasticsearch

3. Запускаємо

```
docker-compose up
```


Створюємо індекс

```
{
  "mappings": {
    "properties": {
      "subject": { "type": "text" },
      "content": { "type": "text" },
      "from": {
        "properties": {
          "email": { "type": "keyword" },
          "name": { "type": "text" }
        }
      },
    },
    "recipients": {
      "properties": {
        "email": { "type": "keyword" },
        "name": { "type": "text" }
      }
    },
    "status": { "type": "keyword" }, // статус повідомлення: очікує відправки, відправлено, помилка
    "timestamp": { "type": "date" }, // час, коли повідомлення поставлене в чергу
    "delayTime": { "type": "long" } // скільки пролежало в черзі до відправки (в мілісекундах)
  },
  "settings": {
    "index": {
      "number_of_shards": 3,
      "number_of_replicas": 1
    }
  }
}
```

Список індексів

GET http://localhost:9200/_cat/indices?v

	health	status	index	uuid	pri	rep	docs.count	docs.deleted	store.size	pri.store.size
1	yellow	open	students	0GeMdwZ-SiSxYQ1qrb5p2A	1	1	102	0	47.2kb	47.2kb
2	yellow	open	notifications-2025	tts27hEFSRSh8-GoVTBJbw	3	1	0	0	675b	675b

Index API

Варіанти додавання документів

PUT /<target>/_doc/<_id>

POST /<target>/_doc/

PUT /<target>/_create/<_id>

POST /<target>/_create/<_id>

В якості Body вказуємо JSON документу

Вставка документу

POST http://localhost:9200/notifications-2025/_doc

```
{
  "subject": "Account settings check",
  "content": "Hello! Check your account settings",
  "from": {
    "email": "admin@profitsoft.ua",
    "name": "ProfITsoft Admin"
  },
  "recipients": [{
    "email": "r.reveguk@profitsoft.ua",
    "name": "Roman Reveguk"
  }],
  "status": "PENDING",
  "timestamp": "2023-02-19T17:00:00"
}
```

Тут ID ми не вказуємо і він генерується автоматично. Так працює швидше, тому що ES не потрібно витратити час на перевірку наявності документу з вказаним ID.

Отримання документу по ID

GET `http://localhost:9200/notifications-2025/_doc/<document_id>`

```
1  {
2    "_index": "notifications-2025",
3    "_id": "0uX9F5sBsEtZYSZispY2",
4    "_version": 1,
5    "_seq_no": 0,
6    "_primary_term": 1,
7    "found": true,
8    "_source": {
9      "subject": "Account settings check",
10     "content": "Hello! Check your account settings",
11     "from": {
12       "email": "admin@profitsoft.ua",
13       "name": "ProfITsoft Admin"
14     },
15     "recipients": [
16       {
17         "email": "r.reveguk@profitsoft.ua",
18         "name": "Roman Reveguk"
19       }
20     ],
21     "status": "PENDING",
22     "timestamp": "2023-02-19T17:00:00"
23   }
24 }
```

Вставка/перезаписування документу з ID

PUT http://localhost:9200/notifications-2025/_doc/<document id>

```
{
  "subject": "Account settings check",
  "content": "Hello! Check your account settings",
  "from": {
    "email": "admin@profitsoft.ua",
    "name": "ProfITsoft Admin"
  },
  "recipients": [{
    "email": "r.reveguk@profitsoft.ua",
    "name": "Roman Reveguk"
  }],
  "status": "SENT",
  "timestamp": "2022-01-22T17:00:00",
  "delayTime": 2500 // скільки пролежало в черзі до відправки (в мілісекундах)
}
```

POST http://localhost:9200/notifications-2025/_search

```
{  
  "query": {  
    "match_all": { }  
  },  
  // сортування  
  "sort": [{  
    "timestamp": "desc"  
  }]  
}
```

```
4  ✓  "_shards": {  
5    "total": 3,  
6    "successful": 3,  
7    "skipped": 0,  
8    "failed": 0  
9  },  
10 ✓  "hits": {  
11   "total": {  
12     "value": 1,  
13     "relation": "eq"  
14   },  
15   "max_score": null,  
16   "hits": [  
17     {  
18       "_index": "notifications-2025",  
19       "_id": "@uX9F5sBsEtZYSZispY2",  
20       "_score": null,  
21       "_source": {  
22         "subject": "Account settings check",  
23         "content": "Hello! Check your account settings",  
24         "from": {  
25           "email": "admin@profitsoft.ua",  
26           "name": "ProfITsoft Admin"  
27         },  
28         "recipients": [  
29           {  
30             "email": "r.reveguk@profitsoft.ua",  
31             "name": "Roman Reveguk"  
32           }  
33         ],  
34         "status": "PENDING",  
35         "timestamp": "2023-02-19T17:00:00"  
36       }  
37     ]  
38   }  
39 }
```

Важливо: документ може бути доступний для пошуку не відразу після створення, на переіндексацію у ES йде певний час (див. [refresh_interval](#)).

Пошук із фільтрацією

POST http://localhost:9200/notifications-2025/_search

```
{
  "query": {
    "range": {      // входження в діапазон
      "timestamp": {
        "gte": "2022-01-22",
        "lt": "2099-01-22"
      }
    }
  }
}
```


Фільтрація по декільком полям

```
{
  "query": {
    "bool": {
      "filter": [
        {
          "range": {
            "timestamp": {
              "gte": "2022-01-20",
              "lt": "2024-01-22"
            }
          }
        },
        {
          "term": {      // повне співпадіння значення
            "status": { "value": "SENT" }
          }
        }
      ]
    }
  }
}
```

```
"hits": [
  {
    "_index": "notifications-2025",
    "_id": "0uX9F5sBsEtZYSZispY2",
    "_score": 1.0,
    "_source": {
      "subject": "Account settings check",
      "content": "Hello! Check your account settings",
      "from": {
        "email": "admin@profitsoft.ua",
        "name": "ProfITsoft Admin"
      },
      "recipients": [
        {
          "email": "r.reveguk@profitsoft.ua",
          "name": "Roman Reveguk"
        }
      ],
      "status": "SENT",
      "timestamp": "2022-01-22T17:00:00",
      "delayTime": 2500
    }
  }
]
```

Текстовий пошук (match query)

POST http://localhost:9200/notifications-2025/_search

```
{
  "query": {
    "match": {
      // поле, за яким шукаємо
      "subject": {
        "query": "account configuration check",
        // operator може бути OR (за замовч.) або AND
        "operator": "OR",
        // скільки слів має співпасти (можна вказувати у %)
        "minimum_should_match": 2
      }
    }
  }
}
```

```
{
  "_shards": {
    "total": 3,
    "successful": 3,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "max_score": 0.5753642,
    "hits": [
      {
        "_index": "notifications-2025",
        "_id": "0uX9F5sBsEtZYSZispY2",
        "_score": 0.5753642,
        "_source": {
          "subject": "Account settings check",
          "content": "Hello! Check your account settings",
          "from": {
            "email": "admin@profitsoft.ua",
            "name": "ProfitSoft Admin"
          }
        },
        "recipients": [
          {

```

Вказуємо які поля вибирати

POST http://localhost:9200/notifications-2025/_search

```
{
  "query": {
    "match_all": { }
  },
  // поля, що будуть у відповіді
  "fields": [
    "subject",
    "timestamp",
    "status"
  ],
  // прибираємо документ із відповіді
  "_source": false
}
```

```
{
  "hits": {
    "total": {
      "value": 1,
      "relation": "eq"
    },
    "max_score": 1.0,
    "hits": [
      {
        "_index": "notifications-2025",
        "_id": "0uX9F5sBsEtZYSZispY2",
        "_score": 1.0,
        "fields": {
          "subject": [
            "Account settings check"
          ],
          "timestamp": [
            "2022-01-22T17:00:00.000Z"
          ],
          "status": [
            "SENT"
          ]
        }
      }
    ]
  }
}
```

Якщо документи достатньо великі, а нам потрібно лиш декілька полів, це буде економити ресурси і трафік між ES і нашим застосунком

Scripting

За допомогою скриптів можна писати вирази, вичисляти значення, робити операції оновлення, в т.ч. з умовною логікою

ES підтримує різні мови написання скриптів, мова за замовчуванням - **painless**.

- Безпека
- Продуктивність
- Простота

Структури скрипту

```
"script": {  
  // мова скрипту, наприклад, painless  
  "lang": "...",  
  // source - сам скрипт, або id - посилання на збережений скрипт  
  "source" | "id": "...",  
  // параметри скрипту (опціонально)  
  "params": { ... }  
}
```

<https://www.elastic.co/guide/en/elasticsearch/reference/current/modules-scripting-using.html>

Приклад скрипту

POST http://localhost:9200/notifications-2025/_search

```
{
  "fields": ["status", "subject"],
  // в секції script_fields вказуємо скрипти
  "script_fields": {
    "month": {
      "script": {
        // скрипт повертає місяць для дати
        "source": "return doc['timestamp'].value.getMonth();",
        "lang": "painless"
      }
    }
  }
}
```

```
"hits": {
  "total": {
    "value": 1,
    "relation": "eq"
  },
  "max_score": 1.0,
  "hits": [
    {
      "_index": "notifications-2025",
      "_id": "0uX9F5sBsEtZYSZispY2",
      "_score": 1.0,
      "fields": {
        "month": [
          "JANUARY"
        ],
        "subject": [
          "Account settings check"
        ],
        "status": [
          "SENT"
        ]
      }
    }
  ]
}
```

Посторінковий запит даних

POST http://localhost:9200/notifications-2025/_search

```
{
  "query": {
    "match_all": { }
  },
  "sort": [{
    "timestamp": "desc"
  }],
  // починаючи з такого елемента пошук
  "from": 100,
  // скільки документів повернути у відповіді (за замовч. 10)
  "size": 20
}
```

Avoid using `from` and `size` to page too deeply or request too many results at once. Search requests usually span multiple shards. By default, you cannot use `from` and `size` to page through more than 10,000 hits. If you need to page through more than 10,000 hits, use the `search_after` parameter instead.

Track total hits

Дозволяє зрозуміти, скільки всього документів задовольняє критерії вибірки, у той час як EL віддає у відповіді лиш **size** документів. Керування цим атрибутом дозволяє досягти необхідного функціоналу/продуктивності.

```
{
  "query": {
    "range": {
      "timestamp": {
        "gte": "2022-01-20",
        "lt": "2029-01-22"
      }
    }
  },
  "size": 20,
  "track_total_hits": true // може бути true/false або числом. Визначає чи буде еластик відстежувати повну
                           кількість документів за запитом.
}
```

Якщо **track_total_hits** - число, то **total** у відповіді буде не більше цього числа (далі документи не будуть скануватися).

За замовч. **track_total_hits** дорівнює **10000**.

Масиви вкладених документів

Mapping (нічим не відрізняється від просто вкладених документів):

```
"recipients": {  
  "properties": {  
    "email": { "type": "keyword" },  
    "name": { "type": "text" }  
  }  
}
```

Поле в документі

```
"recipients": [{  
  "email": "r.reveguk@profitsoft.ua",  
  "name": "Roman Reveguk"  
}, {  
  "email": "i.ivanov@profitsoft.ua",  
  "name": "Ivan Ivanov"  
}]
```

Всередині індексу (з точки зору пошуку) зберігається на зразок цього

```
{  
  "recipients.email" : [ "r.reveguk@profitsoft.ua", "i.ivanov@profitsoft.ua" ],  
  "recipients.name" : [ "Roman Reveguk", "Ivan Ivanov" ]  
}
```


Пошук вкладених документів

POST http://localhost:9200/notifications-2025/_search

```
{
  "query": {
    "bool": {
      "filter": [{
        "term": {
          "recipients.email": { "value": "r.reveguk@profitsoft.ua" }
        }
      }, {
        "match": {
          "recipients.name": { "query": "Ivan Ivanov" }
        }
      }
    ]
  }
}
```

```
{
  "_index": "notifications-2025",
  "_id": "0-U7GJsBsEtZYSZiuJZX",
  "_score": 0.0,
  "_source": {
    "subject": "Account settings check",
    "content": "Hello! Check your account settings",
    "from": {
      "email": "admin@profitsoft.ua",
      "name": "ProfITsoft Admin"
    },
    "recipients": [
      {
        "email": "r.reveguk@profitsoft.ua",
        "name": "Roman Reveguk"
      },
      {
        "email": "i.ivanov@profitsoft.ua",
        "name": "Ivan Ivanov"
      }
    ]
  },
  "status": "PENDING",
  "timestamp": "2023-02-19T17:00:00"
}
```

Тип Nested

Створюємо мапінг з типом `nested`

```
"recipients": {  
  "type": "nested",  
  "properties": {  
    "email": {  
      "type": "keyword"  
    },  
    "name": {  
      "type": "text"  
    }  
  }  
}
```

Внутрішньо вкладені об'єкти індексують кожен об'єкт у масиві як окремий прихований документ, що означає, що кожен вкладений об'єкт можна запитувати незалежно від інших за допомогою вкладеного запиту.

<https://www.elastic.co/guide/en/elasticsearch/reference/current/nested.html>

Nested query

```
{
  "query": {
    "nested": {
      "path": "recipients",
      "query": {
        "bool": {
          "must": [
            { "term": { "recipients.email": "r.veveguk@profitsoft.ua" } },
            { "match": { "recipients.name": "Roman Reveguk" } }
          ]
        }
      }
    }
  }
}
```

Aggregations

```
{
  "aggs": {
    "status_statistics": {
      "terms": {
        "field": "status"
      }
    }
  },
  // вимикаємо набір документів (hits) із відповіді
  "size": 0
}
```

```
"hits": {
  "total": {
    "value": 2,
    "relation": "eq"
  },
  "max_score": null,
  "hits": []
},
"aggregations": {
  "status_statistics": {
    "doc_count_error_upper_bound": 0,
    "sum_other_doc_count": 0,
    "buckets": [
      {
        "key": "PENDING",
        "doc_count": 1
      },
      {
        "key": "SENT",
        "doc_count": 1
      }
    ]
  }
}
```

Date Histogram + sub-aggregations

```
{
  "aggs": {
    "distribution_by_days": {      // назва агрегації (м.б. довільною)
      "date_histogram": {        // розбиває поле з часом на тимчасові проміжки
        "field": "timestamp",
        "calendar_interval": "day"
      },
      "aggs": {
        "status_statistics": {
          "terms": {
            "field": "status"
          }
        }
      }
    },
    "size": 0
  }
}
```

```
    "aggregations": {
      "distribution_by_days": {
        "buckets": [
          {
            "key_as_string": "2022-01-22T00:00:00.000Z",
            "key": 1642809600000,
            "doc_count": 1,
            "status_statistics": {
              "doc_count_error_upper_bound": 0,
              "sum_other_doc_count": 0,
              "buckets": [
                {
                  "key": "SENT",
                  "doc_count": 1
                }
              ]
            }
          }
        ],
        "key_as_string": "2022-01-23T00:00:00.000Z",
        "key": 1642896000000,
        "doc_count": 0,
        "status_statistics": {
          "doc_count_error_upper_bound": 0,
          "sum_other_doc_count": 0,
          "buckets": []
        }
      }
    }
  }
```

Оновлення документу по ID

```
POST http://localhost:9200/notifications-2025/_update/<doc_id>
{
  // оновлення вказаних полів документу
  "doc": {
    "status": "SENT",
    "delayTime": 1000
  }
}
```

Не перезаписує документ як PUT /<target>/_doc/<_id>.

<https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-update.html>

Оновлення за запитом

POST http://localhost:9200/notifications-2025/_update_by_query

```
{  
  "query": {  
    "range": {  
      "timestamp": {  
        "gte": "2022-01-20",  
        "lt": "2099-01-22"  
      }  
    }  
  },  
  "script": {  
    "source": "ctx._source.status='ERROR' ",  
    "lang": "painless"  
  }  
}
```

Видалення документів

Видалення одного документу по ID

DELETE http://localhost:9200/notifications-2025/_doc/<document id>

Видалення декількох документів за запитом

POST http://localhost:9200/notifications-2025/_delete_by_query

```
{
  "query": {
    "match": {
      "recipients.email": "r.reveguk@profitsoft.ua"
    }
  }
}
```

<https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-delete-by-query.html#docs-delete-by-query-api-request>

Bulk API

Виконує множинні операції індексації або видалення в одному запиті, що дозволяє значно пришвидшити виконання операцій над безліччю документів.

Тіло запиту вказується у форматі **newline delimited JSON (NDJSON)**

POST `http://localhost:9200/notifications-2025/_bulk`

```
{ "index" : { "_id" : "ID_1" } }  
  
{ "subject": "Task Bulk email 1", "status": "PENDING", "timestamp":  
"2023-02-21T17:00:00"}  
  
{ "delete" : { "_id" : "ID_TO_DELETE" } }  
  
{ "create" : { "_id" : "ID_2" } }  
  
{ "subject": "Task Bulk email 2", "status": "PENDING", "timestamp":  
"2023-02-21T17:10:00"}
```

// expects the partial doc, upsert, and script and its options are specified on the next line

```
{ "update" : { "_id" : "ID_1" }  
  
{ "doc" : { "status" : "SENT" } }
```

<https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-bulk.html>

Index Aliases

Alias - це псевдонім одного або одночасно декількох індексів.

POST http://localhost:9200/_aliases

```
{
  "actions": [
    {
      "add": {
        "index": "notifications-2025",
        "alias": "notifications"
      }
    }
  ]
}
```

В якості "index" можна вказувати також wildcards (*), наприклад

```
"index": "notifications-*".
```

Оновлення мапінгу в індексах

Додавання нових полів

Важливо. Треба створити мапінг до того як ми створюємо новий документ з цим новим полем.

PUT `http://localhost:9200/notifications-2025/_mapping`

```
{  
  "properties": {  
    "topic": {  
      "type": "keyword"  
    }  
  }  
}
```

Коли потрібно змінити тип поля

1. Створюємо новий індекс з новим мапінгом
2. Робимо `_reindex` в новий індекс
3. Використовуємо `alias`, щоб це було непомітно для застосунку

<https://www.elastic.co/guide/en/elasticsearch/reference/current/indices-put-mapping.html>

Spring Data Elasticsearch

pom.xml

```
<dependency>
```

```
  <groupId>org.springframework.boot</groupId>
```

```
  <artifactId>spring-boot-starter-data-elasticsearch</artifactId>
```

```
</dependency>
```

Configuration

@Configuration

```
public class ElasticsearchConfig extends ElasticsearchConfiguration {
```

```
    @Value("${elasticsearch.address}")
```

```
    private String esAddress;
```

@Override

```
public ClientConfiguration clientConfiguration() {
```

```
    return ClientConfiguration.builder()
```

```
        .connectedTo(esAddress)
```

```
        .build();
```

```
}
```

```
}
```

application.properties:

```
elasticsearch.address=localhost:9200
```

Data classes

```
@Getter
@Setter
@Document(indexName="students")
public class StudentData {
    @Id
    private String id;

    @Field(type = FieldType.Text)
    private String name;

    @Field(type = FieldType.Text)
    private String surname;

    @Field(type = FieldType.Date)
    private Instant birthDate;

    @Field(type = FieldType.Keyword)
    private String group;
    // ...
}
```

CrudRepository

```
/**
```

```
 * See docs here:
```

```
 *
```

```
https://docs.spring.io/spring-data/elasticsearch/docs/current/reference/html/#elasticsearch.query-methods.criteria
```

```
 */
```

```
public interface StudentRepository extends CrudRepository<StudentData, String> {
```

```
}
```

ElasticsearchOperations

```
@Autowired
```

```
private ElasticsearchOperations elasticsearchOperations;
```

```
// create mapping for index
```

```
elasticsearchOperations.indexOps(StudentData.class).createMapping();
```

```
// delete index
```

```
elasticsearchOperations.indexOps(StudentData.class).delete();
```

```
// load by id
```

```
StudentData loaded = elasticsearchOperations.get(studentId, StudentData.class);
```


Integration testing with TestContainers

pom.xml:

```
<dependency>  
  <groupId>org.testcontainers</groupId>  
  <artifactId>testcontainers</artifactId>  
  <version>1.17.6</version>  
  <scope>test</scope>  
</dependency>
```

```
<dependency>  
  <groupId>org.testcontainers</groupId>  
  <artifactId>elasticsearch</artifactId>  
  <version>1.17.6</version>  
  <scope>test</scope>  
</dependency>
```

Container configuration

@TestConfiguration

```
public class TestElasticsearchConfiguration extends ElasticsearchConfiguration {
```

```
    @Bean(destroyMethod = "stop")
```

```
    public ElasticsearchContainer elasticsearchContainer() {
```

```
        ElasticsearchContainer container = new ElasticsearchContainer(
```

```
            "docker.elastic.co/elasticsearch/elasticsearch:8.6.1");
```

```
        container.setEnv(List.of(
```

```
            "discovery.type=single-node",
```

```
            "ES_JAVA_OPTS=-Xms1g -Xmx1g",
```

```
            "xpack.security.enabled=false"));
```

```
        container.start();
```

```
        return container;
```

```
    }
```

```
    // override ClientConfiguration bean
```

```
    @Bean @Override
```

```
    public ClientConfiguration clientConfiguration() {
```

```
        return ClientConfiguration.builder()
```

```
            .connectedTo(elasticsearchContainer().getHttpHostAddress())
```

```
            .build();
```

```
    }}
```

Test Class

```
@SpringBootTest
@ContextConfiguration(classes = {ElasticSampleApplication.class, TestElasticsearchConfiguration.class})
class ElasticSampleApplicationTests {

    @Autowired
    private ElasticsearchOperations elasticsearchOperations;

    // ...

    @Test
    void testCreateStudent() {
        StudentSaveDto student = StudentSaveDto.builder()
            .name("Taras")
            .surname("Shevchenko")
            .group("Group-1")
            .phoneNumbers(List.of("380501234567", "380501112233"))
            .build();

        String studentId = studentService.createStudent(student);

        StudentData loaded = elasticsearchOperations.get(studentId, StudentData.class);
        assertThat(loaded.getName()).isEqualTo(student.getName());

        // ...
    }
}
```

http://localhost:5601

